Models in ASP.NET Core Web API

Models in ASP.NET Core Web API

In ASP.NET Core Web API, models serve as the backbone for representing and managing data within an application. They define the structure of data, apply validation rules to ensure integrity, and establish relationships between entities when working with databases. Models serve as blueprints that guide the flow of data between the client and server, making them essential for handling requests, processing business logic, and persisting information in a reliable and organized manner.

What are Models in ASP.NET Core Web API?

In ASP.NET Core Web API, **Models** are **C# Classes** that define the shape and structure of the data your application works with. They describe how data is structured, how it relates to other data, and how it should be validated before being stored or returned. Models act as the **blueprint** for both database entities and data transfer between the client and server.

Key Features of Models:

The following are the Key Features of Models in ASP.NET Core Web API:

Data Representation (POCOs)

Models are typically **POCOs (Plain Old CLR Objects)**, simple classes that do not depend on any base class or framework inheritance. They define properties that reflect the data structure of your application or API.

These classes form the foundation for handling real-world entities, such as employees, Products, or Orders. For example, a Product model may have properties such as Id, Name, Price, and CategoryId. This makes data easier to organize and work with throughout the application.

```
namespace MyFirstWebAPIProject.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
        public decimal Price { get; set; }
        public string Category { get; set; } = null!;
    }
```

```
}
```

These models are used by EF Core (if enabled) to map to database tables.

Data Validation with Data Annotations

Models can be **annotated with attributes** from the System.ComponentModel.DataAnnotations namespace to apply validation rules. These rules validate incoming data automatically before it reaches business logic.

Common Data Annotations:

- [Required] → ensures a property is not null or empty.
- [StringLength(50)] → restricts text length.
- [Range(1, 100)] → enforces numeric ranges.
- [RegularExpression] → checks formats (e.g., email, phone numbers).

This reduces bugs and ensures the **integrity of incoming data**. For example:

```
using System.ComponentModel.DataAnnotations;
public class Product
{
    public int Id { get; set; }

    [Required]
    [StringLength(100, MinimumLength = 3)]
    public string Name { get; set; } = null!;

    [Range(1, 10000.00)]
    public decimal Price { get; set; }

    [Required]
    public string Category { get; set; } = null!;
}
```

These validations are automatically enforced when [ApiController] is used. You can also manually check the above validation using ModelState.IsValid within the action method of a controller.

Defining Relationships (EF Core Models)

When used with **Entity Framework Core**, models can define relationships such as:

- **One-to-Many**
- **One-to-One**
- **Many-to-Many**

This is done using **Navigation Properties** and foreign key conventions. For example, A Product belongs to one Category, while a Category can have many Products (one-to-many).

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; }

    // One-to-Many
    public List<Product> Products { get; set; }
}

public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }

    public int CategoryId { get; set; } // FK
    public Category Category { get; set; }
}
```

Navigation properties (Category Category, List<Product> Products) allow EF Core to manage relationships and enable LINQ queries across related data. EF Core uses these relationships to generate the correct schema and handle joins.

DTOs (Data Transfer Objects)

DTOs are **not Entity Models** — they are **Lightweight Classes** designed to:

- Models can also act as **DTOs**, which are specialized objects for shaping the data exposed by the API.
- Hide sensitive/internal fields (like passwords or internal IDs) and only return what the client actually needs.
- DTOs improve **security, performance, and clarity** of API responses by sending only required data.

Example:

```
public class ProductDTO
{
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Note: You typically need to use **AutoMapper** or manual mapping to convert between Entity Models and DTOs.

Entity Framework Core Integration

When working with EF Core:

- Models map to database **tables**.
- Properties map to **columns**.
- Annotations (such as [Key] and [ForeignKey]) control schema behavior.

EF Core uses these models to generate SQL commands for **CRUD** (Create, Read, Update, Delete) **operations**.

```
public class ApplicationDbContext : DbContext
{
    public DbSet<Product> Products { get; set; }
}
```

EF Core will map Product to a Products table and generate SQL queries like SELECT * FROM Products.

Real-time Analogy

Think of **Models** as the **blueprint of a house**:

- They define what rooms (properties) exist, how rooms connect (relationships), and the rules for construction (validation, i.e., size, safety rules).
- Entity Framework is like the **Builder** who uses this blueprint to construct the actual house (database tables).
- DTOs are like **sales brochures**; you don't show the wiring and plumbing (sensitive data), only what the customer needs to see.

Creating Models in ASP.NET Core Web API

In ASP.NET Core Web API, **Models** represent the structure of your application's data. Models can technically be placed **anywhere**, but for clarity and maintainability, you can create models:

- In a **Dedicated Models folder** within the project (recommended for clarity).
- In **any Folder/Subfolder** of the project (common in small projects).
- In a **Separate Class Library Project,** such as MyApp.Domain or MyApp.Models (preferred for larger applications, as they keep domain models independent of the API).

By convention, creating a dedicated **Models** folder makes your project organized, readable, and easier to maintain. First, create a folder named 'Models' at the project root directory.

Product Model (Entity)

Now, let us add a basic model to represent the Product information. Right-click on the **Models** folder, add a class file named **Product.cs**, and copy and paste the following code. This is a simple model representing a product. It

contains properties like ID, Name, Price, and Category. It includes validation attributes and a relationship with a **Category** entity.

```
using System.ComponentModel.DataAnnotations;
namespace MyFirstWebAPIProject.Models
{
    public class Product
    {
        public int Id { get; set; }

        [Required]
        [StringLength(100, MinimumLength = 3)]
        public string Name { get; set; } = null!;

        [Range(0.01, 10000.00)]
        public decimal Price { get; set; }

        // Foreign Key for Category
        public int CategoryId { get; set; }

        // Navigation property (Relationship)
        public Category Category { get; set; } = null!;
    }
}
```

Explanation:

- CategoryId: Foreign key that links each Product to a Category.
- Category: navigation property → EF Core uses it to establish the relationship.

Category Model (Entity)

We also create a **Category** entity to define a **one-to-many relationship** (one Category can have many Products). So, create a class file named **Category.cs** within the **Models** folder and copy-paste the following code.

```
using System.ComponentModel.DataAnnotations;
namespace MyFirstWebAPIProject.Models
{
    public class Category
    {
        public int Id { get; set; }

        [Required]
        [StringLength(50)]
        public string Name { get; set; } = null!;
```

```
        // Navigation property (One-to-Many relationship)
        public ICollection<Product>? Products { get; set; }
    }
}
```

Explanation:

- The Products collection represents the **one-to-many** relationship (1 Category → many Products).

What are DTOs?

DTOs (**Data Transfer Objects**) are simple C# classes used to transfer data between the client and server in an ASP.NET Core Web API. They act as a **contract** that defines exactly what data should be sent or received, without exposing the full database entities.

By using DTOs, you can hide sensitive fields, reduce unnecessary data transfer, enforce validation rules, and shape responses in a client-friendly way, making your API more secure, efficient, and maintainable.

Do We Need Different DTOs for Create, Update, and Retrieve Operations?

Yes, it is **highly recommended** to create **separate DTOs (Data Transfer Objects)** for different CRUD operations, especially for **Create (POST)**, **Update (PUT/PATCH)**, and **Retrieve (GET)**. This design pattern ensures clarity, maintainability, and validation precision, aligning with the **Single Responsibility Principle**.

Each operation has different requirements in terms of the data it expects or returns. Using dedicated DTOs helps you avoid exposing internal or sensitive fields, simplifies validation logic, and makes your API more robust and scalable.

Create DTO (for POST operations)

A **Create DTO** is used when a new resource needs to be added to the system. Since the database usually generates the ID, this DTO contains only the required properties to create a record. It ensures that only the necessary fields are submitted by the client and applies validation rules to prevent insufficient data from being inserted.

- Excludes the Id property (auto-generated by the database).
- Contains required fields such as Name, Price, and Category ID.
- Enforces validation rules using **data annotations**.
- Prevents over-posting by limiting input only to fields needed for creation.

Update DTO (for PUT/PATCH operations)

An **Update DTO** is used when modifying an existing resource. Unlike the Create DTO, it includes the Id so the API knows which record to update. It may also allow partial updates (with PATCH) or require all fields (with PUT). This separation ensures that updates are intentional and validated.

- Includes the Id property to identify the resource.
- Contains fields that can be updated, such as Name, Price, and CategoryId.
- Can support full updates (PUT) or partial updates (PATCH).
- Ensures only valid data is sent for modifications.

Retrieve DTO (for GET operations)

A **Retrieve DTO** is used to shape the data returned to the client. Instead of exposing the full entity with all fields (including sensitive or unnecessary ones), this DTO returns only what the client should see. It can also transform data for better readability, such as showing CategoryName instead of CategoryId.

- Includes the ID and client-friendly fields.
- Excludes sensitive or internal properties (e.g., cost price, audit fields).
- Can flatten relationships (e.g., return CategoryName instead of a complete Category object).
- Provides a clean and optimized data structure for API responses.

First, create a folder named **'DTOs'** in the project root directory, where we will store all our DTOs.

Creating ProductDTO

This DTO is used for returning product details. It excludes sensitive information and only exposes the properties required by the client. Create a class file named **ProductDTO.cs** within the **DTOs** folder and copy and paste the following code.

```
namespace MyFirstWebAPIProject.DTOs
{
    public class ProductDTO
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
        public decimal Price { get; set; }
        public string CategoryName { get; set; } = null!;
    }
}
```

Explanation:

- ProductDTO includes **only the necessary fields** for API responses.
- Instead of sending the complete Category entity, it exposes CategoryName.

- This ensures that sensitive or unnecessary data isn't exposed to the client.

ProductCreateDTO

This DTO is used when creating a new product. It **excludes the Id** (since the database will generate it) and only includes the properties needed to create a product. Create a class file named **ProductCreateDTO.cs** within the **DTOs** folder and then copy and paste the following code.

```
using System.ComponentModel.DataAnnotations;
namespace MyFirstWebAPIProject.DTOs
{
    public class ProductCreateDTO
    {
        [Required(ErrorMessage = "Product name is required.")]
        [StringLength(100, MinimumLength = 3, ErrorMessage = "Product name must
be between 3 and 100 characters.")]
        public string Name { get; set; } = null!;

        [Range(0.01, 10000.00, ErrorMessage = "Price must be between 0.01 and
10,000.")]
        public decimal Price { get; set; }

        [Required(ErrorMessage = "CategoryId is required.")]
        public int CategoryId { get; set; }
    }
}
```

Explanation:

- Excludes Id since it is auto-generated.
- Includes essential fields required to create a product (Name, Price, CategoryId).
- CategoryId ensures the new product is linked to a valid category.
- Keeps the DTO **lightweight and focused** only on creation.

ProductUpdateDTO

This DTO is used when updating an existing product. It **includes the ID** so the API knows which record to update. Create a class file named **ProductUpdateDTO.cs** within the **DTOs** folder and then copy and paste the following code.

```
using System.ComponentModel.DataAnnotations;
namespace MyFirstWebAPIProject.DTOs
{
    public class ProductUpdateDTO
```

```
    {
        [Required(ErrorMessage = "Product Id is required.")]
        public int Id { get; set; }

        [Required(ErrorMessage = "Product name is required.")]
        [StringLength(100, MinimumLength = 3, ErrorMessage = "Product name must
be between 3 and 100 characters.")]
        public string Name { get; set; } = null!;

        [Range(0.01, 10000.00, ErrorMessage = "Price must be between 0.01 and
10,000.")]
        public decimal Price { get; set; }

        [Required(ErrorMessage = "CategoryId is required.")]
        public int CategoryId { get; set; }
    }
}
```

Explanation:

- Includes ID to identify the record being updated.
- Allows modification of product details (Name, Price, CategoryId).
- Suitable for both **PUT (full update)** and **PATCH (partial update with adjustments)**.
- Keeps update logic clear and separate from creation and retrieval.

Using Models and DTOs in Controller:

In ASP.NET Core Web API, **Controllers** handle HTTP requests and responses, and **models** represent the data being exchanged. When combined with **DTOs**, this creates a clean and secure data flow:

- **GET** → Retrieve model/DTO data and return it to the client.
- **POST** → Accept model/DTO data from the request body to create new records.
- **PUT** → Accept model/DTO data for updating existing records.
- **DELETE** → Accept an identifier to delete the correct record.

By introducing **DTOs**, we separate internal entity representation (database structure) from client-facing data contracts, ensuring security and flexibility.

Creating Products Controller

Let's create a Products Controller to manage Product entities. Right-click on the Controllers folder, select **Add > Controller,** choose **API Controller – Empty**, name it **ProductsController,** and then copy and paste the following code.

```
using Microsoft.AspNetCore.Mvc;
using MyFirstWebAPIProject.Models;
using MyFirstWebAPIProject.DTOs;

namespace MyFirstWebAPIProject.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductsController : ControllerBase
    {
        // Hardcoded Categories
        private static List<Category> _categories = new List<Category>
        {
            new Category { Id = 1, Name = "Electronics" },
            new Category { Id = 2, Name = "Furniture" },
        };

        // Hardcoded Products
        private static List<Product> _products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 1000.00m, CategoryId =
1 },
            new Product { Id = 2, Name = "Desktop", Price = 2000.00m, CategoryId
= 1 },
            new Product { Id = 3, Name = "Chair", Price = 150.00m, CategoryId = 2
},
        };

        // GET: api/products
        [HttpGet]
        public ActionResult<IEnumerable<ProductDTO>> GetProducts()
        {
            var productDTOs = _products.Select(p => new ProductDTO
            {
                Id = p.Id,
                Name = p.Name,
                Price = p.Price,
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
p.CategoryId)?.Name ?? "Unknown"
            }).ToList();

            return Ok(productDTOs);
        }

        // GET: api/products/{id}
        [HttpGet("{id}")]
        public ActionResult<ProductDTO> GetProduct(int id)
```

```
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product == null)
            {
                return NotFound(new { Message = $"Product with ID {id} not
found." });
            }

            var productDTO = new ProductDTO
            {
                Id = product.Id,
                Name = product.Name,
                Price = product.Price,
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
product.CategoryId)?.Name ?? "Unknown"
            };

            return Ok(productDTO);
        }

        // POST: api/products
        [HttpPost]
        public ActionResult<ProductDTO> PostProduct([FromBody] ProductCreateDTO
createDto)
        {
            var newProduct = new Product
            {
                Id = _products.Max(p => p.Id) + 1,
                Name = createDto.Name,
                Price = createDto.Price,
                CategoryId = createDto.CategoryId
            };

            _products.Add(newProduct);

            var productDTO = new ProductDTO
            {
                Id = newProduct.Id,
                Name = newProduct.Name,
                Price = newProduct.Price,
                CategoryName = _categories.FirstOrDefault(c => c.Id ==
newProduct.CategoryId)?.Name ?? "Unknown"
            };

            return CreatedAtAction(nameof(GetProduct), new { id = productDTO.Id
}, productDTO);
        }
```

```
        // PUT: api/products/{id}
        [HttpPut("{id}")]
        public IActionResult UpdateProduct(int id, [FromBody] ProductUpdateDTO
updateDto)
        {
            if (id != updateDto.Id)
            {
                return BadRequest(new { Message = "ID mismatch between route and
body." });
            }

            var existingProduct = _products.FirstOrDefault(p => p.Id == id);
            if (existingProduct == null)
            {
                return NotFound(new { Message = $"Product with ID {id} not
found." });
            }

            // Update product
            existingProduct.Name = updateDto.Name;
            existingProduct.Price = updateDto.Price;
            existingProduct.CategoryId = updateDto.CategoryId;

            return NoContent();
        }

        // DELETE: api/products/{id}
        [HttpDelete("{id}")]
        public IActionResult DeleteProduct(int id)
        {
            var product = _products.FirstOrDefault(p => p.Id == id);
            if (product == null)
            {
                return NotFound(new { Message = $"Product with ID {id} not
found." });
            }

            _products.Remove(product);
            return NoContent();
        }
    }
}
```

Understanding Action Methods:

Let us understand the use of each action method:

GetProducts() – Retrieve All Products

Handles GET /api/products requests.

- Retrieves the full list of products from the in-memory _products collection.
- Maps each Product entity into a ProductDTO (so only safe, necessary fields are exposed).
- Returns the list inside an HTTP 200 OK response.
- **Use case**: Used by clients to display a product catalog.

GetProduct(int id) – Retrieve a Single Product by ID

Handles GET /api/products/{id} requests.

- Searches _products for a product with the given id.
- If not found → returns 404 Not Found with a descriptive message.
- If found → maps it to a ProductDTO and returns 200 OK.
- **Use case**: Used by clients to view details of a specific product (e.g., clicking on a product card).

PostProduct(ProductCreateDTO createDto) – Create a New Product

Handles POST /api/products requests.

- Accepts a ProductCreateDTO from the request body.
- Creates a new Product entity and assigns a new ID (simulating database auto-increment).
- Adds the new product to the _products array.
- Maps the new entity to a ProductDTO for the response.
- Returns 201 Created with the new product's details and a Location header pointing to the new resource.
- **Use case**: Used by clients to add a new product (e.g., admin adds a new item to inventory).

UpdateProduct(int id, ProductUpdateDTO updateDto) – Update an Existing Product

Handles PUT /api/products/{id} requests.

- Checks that the ID in the route matches the updateDto.Id in the request body (to prevent mismatches).
- Searches for the existing product.
- If not found → returns 404 Not Found.
- If found → updates the product's properties (Name, Price, CategoryId).
- Returns 204 No Content (success but no response body).
- **Use case**: Used by clients/admins to update an existing product's details (e.g., change price or category).

DeleteProduct(int id) – Remove a Product

Handles DELETE /api/products/{id} requests.

- Looks up the product by id.
- If not found → returns 404 Not Found.
- If found → removes it from _products.
- Returns 204 No Content (success with no body).
- **Use case**: Used by admins to remove discontinued products from the catalog.

Testing the Products Controller APIs using .HTTP File:

You can test the APIs in various ways, such as using Postman, Fiddler, and Swagger. However, .NET 8 provides the .http file, and by using that file, we can also test the functionality.

What is the .http File in .NET?

- When you create an ASP.NET Core Web API project in Visual Studio, it automatically generates a .http file.
- This .http file name will be the same name as your project name. My Project name is MyFirstWebAPIProject, so Visual Studio creates the MyFirstWebAPIProject.http file.
- This file contains sample HTTP requests that you can run directly inside the IDE.
- Each request block starts with ###, and you will see a **Send Request** link above it to trigger the request.
- The IDE sends the request to your running API and displays the response inline, eliminating the need to switch to Postman.

Modifying MyFirstWebAPIProject.http file

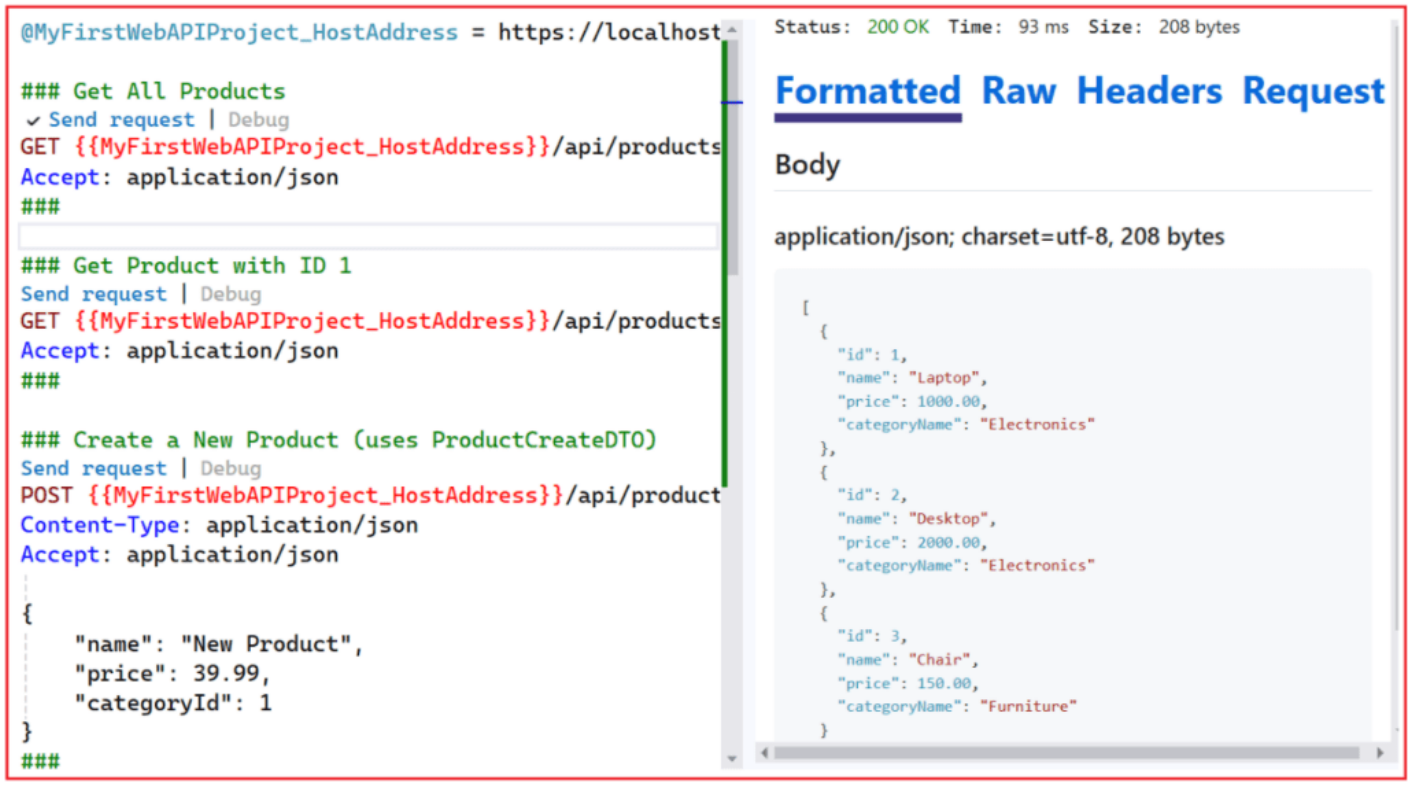
Please open the **MyFirstWebAPIProject.http** file and copy and paste the following code. Please change the port number with the port number on which your application is running.

```
@MyFirstWebAPIProject_HostAddress = https://localhost:7191

### Get All Products
GET {{MyFirstWebAPIProject_HostAddress}}/api/products
Accept: application/json
###

### Get Product with ID 1
GET {{MyFirstWebAPIProject_HostAddress}}/api/products/1
Accept: application/json
###

### Create a New Product (uses ProductCreateDTO)
POST {{MyFirstWebAPIProject_HostAddress}}/api/products
Content-Type: application/json
```

```
Accept: application/json

{
    "name": "New Product",
    "price": 39.99,
    "categoryId": 1
}
###

### Update Product with ID 1 (uses ProductUpdateDTO)
PUT {{MyFirstWebAPIProject_HostAddress}}/api/products/1
Content-Type: application/json
Accept: application/json

{
    "id": 1,
    "name": "Updated Product",
    "price": 49.99,
    "categoryId": 1
}
###

### Delete Product with ID 1
DELETE {{MyFirstWebAPIProject_HostAddress}}/api/products/1
Accept: application/json
###
```

Testing:

Before testing, ensure your API runs locally and listens on the correct port. Open the .http file in your IDE, and you should see "**Send Request**" links above each HTTP request. Click these links to execute the requests, and you will see the responses directly in your IDE, as shown in the image below.



Notes for Testing

- **Run your API** first (dotnet run or F5 in Visual Studio).
- **Check your port** in launchSettings.json and replace 7191 with your actual port.
- **Payload format**:
  - **Create (POST)** → Must match ProductCreateDTO → No Id, just Name, Price, CategoryId.
  - **Update (PUT)** → Must match ProductUpdateDTO → Includes Id, Name, Price, CategoryId.
- **Swagger** → You can also use Swagger UI (/swagger) for testing, but .http files are faster for quick checks.

Real-time Analogy: Think of the .http file like a **notebook of ready-made forms**. Instead of filling out Postman every time, you just click “Send Request” in your notebook, and the request goes directly to your API.

Models in ASP.NET Core Web API not only represent the shape of data but also enforce validation, manage relationships, and optimize how data is shared with clients. When used alongside DTOs and controllers, they create a clean separation of concerns, making applications more reliable, scalable, and easier to maintain. In essence, models act as the blueprint for building well-structured and efficient Web APIs.

DOT NET TUTORIALS

Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET

Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

2 thoughts on “Models in ASP.NET Core Web API”

KRISHNA
MAY 25, 2024 AT 10:26 PM

Actually the put method is not working . could you please look into it.
Errornumber:Compiler Error CS0161

Reply

SAMEERA
JANUARY 31, 2025 AT 3:33 PM

Nice

Reply